Student Marks Matrix Analysis

Python Program

Assigned syllabus matter: Python solutions using only the assigned syllabus: computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, nested lists, tuples, tuple operations, and unpacking.

```
# Student Marks Matrix Analysis

students = int(input("Enter number of students: "))
subjects = int(input("Enter number of subjects: "))

marks = []

print("\\nEnter marks:")
for i in range(students):
    row = []
    print(f"\\nStudent {i + 1}")

    for j in range(subjects):
        mark = int(input(f"Subject {j + 1}: "))
        row.append(mark)

    marks.append(row)

# Row-wise Statistics (Student Performance)
print("\\nStudent Statistics")

top_student = 0
highest_total = 0

for i in range(students):
    total = sum(marks[i])
    average = total / subjects

    print(f"Student {i + 1}: Total = {total}, Average = {average:.2f}")

    if total > highest_total:
        highest_total = total
        top_student = i + 1

# Column-wise Statistics (Subject Performance)
print("\\nSubject Statistics")

low_average_subjects = []

for j in range(subjects):
    subject_total = 0

    for i in range(students):
        subject_total += marks[i][j]

    subject_average = subject_total / students

    print(f"Subject {j + 1}: Average = {subject_average:.2f}")

    if subject_average < 40:
        low_average_subjects.append(j + 1)

# Top Performer
print(f"\\nTop Performer: Student {top_student}")
print(f"Highest Total Marks: {highest_total}")

# Low Performing Subjects
if len(low_average_subjects) > 0:
    print("\\nSubjects with Low Average Performance:")

    for subject in low_average_subjects:
        print(f"Subject {subject}")
else:
    print("\\nNo subjects have low average performance.")
```